



CAPSTONE PROJECT DOCUMENTATION

Computer Science · Summer 2026 Virtual Showcase

Project name: StudyBros

Team members: Kirneill Joseph, Daniel Labrador, Goncalo F Ferreira Nascimento, Lenyn Gabriel Murillo Holguin

Team leader: Kirneill Joseph

Product owner: Lenyn Gabriel Murillo Holguin

Instructor: Prof. Masoud Sadjadi, Knight Foundation School of Computing and Information Sciences, FIU

Date: 07/29/2026

Contents

Contents	2
1 Executive Summary	3
2 Problem & Stakeholders (SO1)	3
2.1 Problem Statement	3
2.2 Stakeholders & User Needs	3
2.3 Constraints & Assumptions	3
3 Requirements Specification (SO1, SO2)	4
3.1 Functional Requirements	4
3.2 Non-Functional Requirements	6
4 Solution Design & Architecture (SO2)	7
4.1 Solution Overview	7
4.2 Architecture	7
5 Implementation (SO2, SO6)	8
6 Testing, Verification & Evaluation (SO2, SO6)	9
7 Software Engineering Evidence (SO6)	9
7.1 Software Development Life Cycle	11
7.2 Project Specification	11
7.3 Feasibility Study and Project Plan	12
7.4 System Walkthroughs	12
8 Professional, Ethical & Legal Considerations (SO4)	12
9 Team & Process (SO5)	12
10 Communication Artifacts (SO3)	13
Appendix A — Student Outcomes & CS Evidence Map	15
Appendix B — Revision History	15

1 Executive Summary

Studying from lecture material is a workflow problem before it's a content problem. Students have PDFs, slide decks, and recorded lectures. What they don't have is time to manually turn that material into flashcards, quizzes, and practice tests, or a reliable way to know what they've actually mastered versus what they've just seen once.

StudyBros (originally built as KSpider) solves this by ingesting learning material directly (PDF, PPTX, TXT, MD, or recorded video/audio) and using AI to generate flashcards, multiple-choice quizzes, mixed-format practice tests, and audio-ready summaries automatically. What started as a Phase 1 CLI tool has grown into a full study platform this semester: it now schedules reviews using the FSRS v5 spaced repetition algorithm, tracks depth of understanding with Bloom's taxonomy instead of just recall, and reports mastery, streaks, and strengths/weaknesses per topic.

This semester's build shipped three ways to use the system, not one. A FastAPI backend and Next.js web app give students a full browser experience; upload, generate, study, track progress. An MCP server exposes the same ingestion, generation, and review pipeline to any MCP-compatible LLM client (Claude, ChatGPT, Cursor), so a student can use the tool with zero OpenAI API key and zero new habits. And the original CLI remains available for direct, scriptable use. Generation itself is provider-agnostic (OpenAI, Anthropic, or OpenRouter) selectable per request, with a fallback that lets a user supply their own key in-browser if the server doesn't have one configured.

The headline result: a working, end-to-end platform with 200+ passing backend tests, clean lint/typecheck across the codebase, and live-verified upload, generation, review, and progress flows — built by a four-person team after losing a member mid-project, on top of an already-complete Phase 1 MVP.

2 Problem & Stakeholders (SO1)

2.1 Problem Statement

Students constantly deal with tight schedules and scattered study habits. Course material is spread across PDFs, PowerPoints, text files, and occasionally JSON — and turning that raw material into something you can actually study from (flashcards, quizzes, practice tests) takes time most students don't have. The gap isn't a lack of material, it's the workflow: going from "I have the slides" to "I have structured practice questions" is manual, repetitive, and gets skipped when time is short.

KSpider closes that gap. It ingests lecture material in whatever format it's already in, and uses AI to generate ready-to-use study assets automatically — no manual transcription, no manually writing quiz questions.

2.2 Stakeholders & User Needs

Type	Need
Students	Convert lengthy lecture PDFs or slides to save up time
Self-learners	Generate quizzes from downloaded materials
Instructors	Generate ready-to-go practice materials
Professionals	Generate structured documentation

2.3 Constraints & Assumptions

Technical constraints:

- Python 3.11+ required; the codebase targets 3.11 and 3.12 specifically
- SQLite chosen for zero-infrastructure local storage, with SQLAlchemy 2.0 abstracting the connection so a future move to PostgreSQL only requires a connection-string change
- FFmpeg is a required system dependency for any video/audio ingestion; not bundled, must be installed separately on the host OS
- Generation depends on a third-party LLM provider (OpenAI, Anthropic, or OpenRouter). In CLI mode, a valid API key is required and billed per call; in MCP mode, no API key is needed since the connected LLM client (Claude, ChatGPT, Cursor, etc.) handles generation directly
- Cost guardrails are enforced before every generation call: MAX_CHUNKS_PER_GENERATION (default 5) and MAX_TOKENS_PER_RESPONSE (default 4,000), both configurable via environment variables

Schedule constraints:

- Fixed capstone deadline; documentation and showcase submission due Wednesday, July 29, 5:00 PM ET

Resource/organizational constraints:

- Three parallel delivery surfaces were scoped (MCP server, LLM Skills/docs, and a Next.js web app on Vercel), which spread a four-person team thin across backend, frontend, and infra work simultaneously
- Deployment target assumes Vercel (frontend) + Railway or Render (backend); final hosting decision was still open as of the last work session

Assumptions:

- Users have, or are willing to obtain, their own LLM API key if not using the keyless MCP path
- Source material is primarily text-extractable (PDF/PPTX/TXT/MD) or transcribable audio/video; the system assumes reasonably well-formed input files
- MIT license was assumed appropriate for an academic/open-source capstone deliverable

3 Requirements Specification (SO1, SO2)

3.1 Functional Requirements

FR1	The system SHALL accept files in formats: .pptx, .pdf, .txt, .md, .mp4, .mov, .webm
FR2	The system SHALL compute SHA-256 hashes on ingest for duplicate detection
FR3	The system SHALL extract raw text from document formats (PPTX, PDF, TXT, MD)
FR4	The system SHALL transcribe audio/video files via OpenAI Whisper, chunking audio into ≤25MB segments to respect the Whisper API limit
FR5	The system SHALL store extracted content and metadata in a persistent SQLite database
FR6	The system SHALL support both single-file and multi-file (batch) upload through the web app, with drag-and-drop and sequential upload progress
FR7	The system SHALL support document rename and bulk delete from the web app
FR8	The system SHALL return stage-specific diagnostic errors on ingestion failure (save, scan, extractor selection, hash, extract, deduplicate, persist) rather than a generic failure message
FR9	The system SHALL chunk documents into semantically coherent segments

FR10	Chunking SHALL prioritize paragraph/heading boundaries over arbitrary splits
FR11	The system SHALL fall back to fixed-size chunking with overlap when document structure is absent
FR12	The system SHALL filter out chunks below a minimum character threshold
FR13	The system SHALL generate flashcard sets (question, answer, tags, difficulty)
FR14	The system SHALL generate multiple-choice quizzes with explanations
FR15	The system SHALL generate mixed-type practice tests (multiple-choice, true/false, short answer)
FR16	The system SHALL generate audio-friendly summaries for TTS consumption
FR17	Generation SHALL use schema-validated structured outputs (Pydantic v2) so responses never require regex-parsing of freeform LLM text
FR18	The system SHALL allow the user to specify item count per generation call
FR19	The system SHALL support multiple LLM providers for generation — OpenAI, Anthropic, and OpenRouter — selectable per request
FR20	The system SHALL support generation with no server-side API key required, via an MCP server that delegates generation to the connected LLM client
FR21	When no server-side key is configured for a provider, the web app SHALL prompt the user for a personal API key and use it for that request only
FR22	The system SHALL schedule flashcard reviews using the FSRS v5 algorithm, based on a power-law forgetting curve
FR23	The system SHALL accept a 1–4 review rating (Again/Hard/Good/Easy) per card and update that card's stability and difficulty parameters accordingly
FR24	The system SHALL compute and surface cards due for review, sorted by urgency (lowest retrievability first)
FR25	The system SHALL track understanding depth per topic using Bloom's taxonomy levels, not recall alone
FR26	The system SHALL evaluate topic mastery against defined criteria: >90% retrievability at >30-day review intervals, >85% accuracy, and Bloom Level 3+ understanding
FR27	The system SHALL persist completed study sessions (flashcard, quiz, practice test) with item counts, correct counts, and Bloom distribution for analytics
FR28	The system SHALL report per-topic strengths and weaknesses across all studied material
FR29	The system SHALL track study consistency (streaks, rolling-window percentage) from session history
FR30	The system SHALL award competency achievements based on defined mastery and consistency milestones
FR31	The system SHALL reconstruct progress records from historical review data when direct progress records are missing (backfill)
FR32	The system SHALL export study sets to JSON (canonical format with metadata)

FR33	The system SHALL export flashcards to Anki-compatible CSV (semicolon-delimited)
FR34	The system SHALL export quizzes and tests to formatted Markdown
FR35	The system SHALL provide a FastAPI REST backend exposing all core operations
FR36	The system SHALL provide a Next.js web frontend for upload, browsing, generation, study, and progress review
FR37	The system SHALL provide an MCP server exposing ingestion, generation, review, and progress tools/resources/prompts to any MCP-compatible LLM client, requiring no API key
FR38	The system SHALL provide a CLI for ingest, generate, export, list, and status operations
FR39	The web UI SHALL provide an interactive flashcard viewer with flip animation and an in-browser quiz/test player

3.2 Non-Functional Requirements

NFR1	Ingestion SHALL complete in under 5 seconds per document
NFR2	Generation calls SHALL complete in under 30 seconds end-to-end
NFR3	The system SHALL NOT lose data or crash unrecoverably during normal operation
NFR4	Upload and generation failures SHALL surface a diagnosable, stage-specific error (save, scan, extractor selection, hash, extract, deduplicate, persist) rather than a generic server error
NFR5	The system SHALL perform duplicate detection via SHA-256 hash comparison before inserting a new source
NFR6	The system SHALL enforce hard, environment-configurable limits before every generation call — default max 5 chunks per generation and 4,000 tokens per response
NFR7	The system SHALL run on Windows, Linux, and macOS
NFR8	The system SHALL support Python 3.11 and 3.12
NFR9	API keys SHALL be loaded from a server-side .env file when configured server-side
NFR10	User-supplied API keys entered in the browser SHALL be scoped to that request only and SHALL NOT be persisted server-side
NFR11	The codebase SHALL pass ruff linting and mypy type-checking cleanly before merge
NFR12	Changes SHALL be covered by the automated regression suite (200+ backend tests) run before merging
NFR13	The generation layer SHALL be provider-agnostic — adding a new LLM provider SHALL NOT require changes to the ingestion, chunking, or export layers

4 Solution Design & Architecture (SO2)

4.1 Solution Overview

The core ingest → chunk → generate → export pipeline from the original KSpider design carries over unchanged; it works, and there was no reason to rebuild it. This semester's work extended that pipeline in three directions instead of replacing it:

1. **Learning science layer:** FSRs v5 spaced repetition scheduling and Bloom's-taxonomy-based mastery tracking were added on top of the existing StudySet model, rather than folding review logic into generation. Reviews, sessions, and progress are tracked as their own entities so scheduling logic stays independent of how a study set was generated.
2. **Multi-surface delivery:** the same core library now backs three separate front doors — a CLI, a FastAPI + Next.js web app, and an MCP server. The design decision was to keep study_guide as a provider-agnostic core library with zero UI assumptions, so the API layer, MCP layer, and CLI are all thin wrappers around the same ingestion/generation/learning modules rather than three separate implementations.
3. **Provider-agnostic generation:** OpenAI's beta.chat.completions.parse() schema-validated approach was generalized into a provider interface supporting OpenAI, Anthropic, and OpenRouter, so a user (or the MCP host LLM) is never locked to one vendor.

The team deliberately kept SQLite rather than introducing a hosted database — the original reasoning (zero infra dependency, ORM abstraction covers future migration) still held, and a four-person team already juggling three delivery surfaces didn't have spare bandwidth to also stand up and manage separate database infrastructure.

4.2 Architecture

AI STUDY GUIDE "StudyBros"					
FRONTEND	Web UI (Next.js): Upload · Documents · Generate · Study · Quiz/Test player · Progress · Achievements CLI (Click): ingest, generate, export, list, status MCP Client: any MCP-compatible LLM (Claude, ChatGPT, Cursor)				
API	FastAPI REST (api/routes/): documents.py upload, list, rename, delete, bulk-delete, chunks generate.py flashcards, quiz, practice-test, summary, providers study.py review, due, schedule, session, mastery, progress study_sets.py list, detail, delete gamification.py phase, achievements, strengths-weaknesses, consistency export.py export by study set MCP Server (mcp_server/server.py): 10 tools · 9 resources · 8 prompts – mirrors the REST surface for MCP clients, generation delegated to the connected LLM (no key needed)				
APPLICATION	<table> <tr> <td>Ingestion: FileScanner, Extractors (PPTX/PDF/TXT/MD/Video), TextChunker</td><td>Learning: FSRS Scheduler (scheduler.py) Gamification engine (gamification.py)</td></tr> <tr> <td>Generation: Schemas (Pydantic v2), Prompts, Generator (OpenAI/Anthropic/OpenRouter)</td><td>Export: JSONExporter, AnkiExporter, MarkdownExporter</td></tr> </table>	Ingestion: FileScanner, Extractors (PPTX/PDF/TXT/MD/Video), TextChunker	Learning: FSRS Scheduler (scheduler.py) Gamification engine (gamification.py)	Generation: Schemas (Pydantic v2), Prompts, Generator (OpenAI/Anthropic/OpenRouter)	Export: JSONExporter, AnkiExporter, MarkdownExporter
Ingestion: FileScanner, Extractors (PPTX/PDF/TXT/MD/Video), TextChunker	Learning: FSRS Scheduler (scheduler.py) Gamification engine (gamification.py)				
Generation: Schemas (Pydantic v2), Prompts, Generator (OpenAI/Anthropic/OpenRouter)	Export: JSONExporter, AnkiExporter, MarkdownExporter				
DATABASE	sources · documents · chunks · study_sets card_reviews · study_sessions · user_progress · achievements · gamification_state				
EXTERNAL	OpenAI GPT / Anthropic Claude / OpenRouter – generation OpenAI Whisper – audio/video transcription FFmpeg – audio extraction				

The ingestion and generation pipeline flows (FileScanner → Extractor → TextChunker → SQLite; and Load chunks → Build prompt → Provider API → Pydantic validation → Save study set) are unchanged from the original design. What's new is what happens after a study set exists:

- A **review flow** (POST /study/{id}/review) takes a 1–4 FSRs rating per card, updates that card's stability/difficulty/retrievability, and upserts a user_progress row for the topic.
- A **session flow** (POST /study/{id}/session) persists a completed flashcard/quiz/test session (item count, correct count, Bloom distribution) so consistency and phase analytics have real session data instead of only individual card reviews.
- A **backfill path** (GET /study/progress) reconstructs user_progress from historical card_reviews when a user has review history but no progress rows yet — this covers users whose data predates the progress-tracking feature.

Entity relationships: sources → documents → chunks and documents → study_sets are unchanged from Phase 1. New tables hang off study_sets and documents: card_reviews references study_sets (per-card FSRs state), study_sessions references study_sets (session-level results), and user_progress references documents and is keyed by topic_tag (per-topic mastery). achievements and gamification_state are global, not tied to a specific document.

5 Implementation (SO2, SO6)

Core library	Python 3.11+, SQLAlchemy 2.0, Pydantic v2, Click
Backend API	FastAPI, Uvicorn, python-multipart
MCP Server	FastMCP
Frontend	Next.js 16, React 19, TypeScript 5, Tailwind CSS 4, Framer Motion
Generator Providers	OpenAI (openai), Anthropic (anthropic), OpenRouter
Transcription	OpenAI Whisper, FFmpeg
Extraction	python-pptx, pypdf
Database	SQLite
Testing	pytest, pytest-cov, pytest-asyncio, ruff, mypy, ESLint

Phase 1 (delivered previously) covered the ingestion pipeline, chunking, generation, export, database, and CLI — all of that carried forward unchanged. This semester's work added:

- **FastAPI backend** (api/) — six route modules (documents, generate, study, study_sets, gamification, export) exposing the full pipeline over REST, with Pydantic response models enforcing a typed contract against the frontend
- **Next.js web app** (frontend/) — 12 routes covering upload, document management, generation, flashcard/quiz/test study flows, progress, achievements, and settings, built with a dark-themed design system (accent teal #00D4AA, ivory text, #1A1A2E background)
- **MCP server** (mcp_server/) — 10 tools, 9 resources, and 8 prompts mirroring the REST surface for any MCP-compatible LLM client, so generation can run with zero server-side API key

- **Learning science module** (`study_guide/learning/`) — FSRS v5 scheduler, gamification engine (phase tracking, achievements, streaks), and five new database models (CardReview, StudySession, UserProgress, Achievement, GamificationState)
- **Multi-provider generation** — provider abstraction supporting OpenAI, Anthropic, and OpenRouter, with per-request key override and a browser-side key modal for users without a server-configured key
- **Upload hardening** — multi-file drag-and-drop upload, stage-specific failure diagnostics, encrypted/unreadable PDF detection, and a persistent upload diagnostics log

The layered design held up under implementation without requiring rework: `study_guide/` stayed a UI-agnostic core library throughout, and the API layer, MCP layer, and CLI all import from it directly rather than duplicating logic. Some deviations found during implementation had to be corrected mid-semester and are worth documenting as evidence of the design-to-implementation feedback loop:

- The `/gamification/consistency` endpoint originally returned an untyped domain dict, which drifted from what the frontend expected and caused a runtime crash. The fix was architectural, not cosmetic; a `ConsistencyResponse` Pydantic model was added so the API contract is enforced at the type level going forward, not just checked by convention.
- Study set content (`content_json`) is stored as structured wrapper objects (e.g., `{"cards": [...]}`), but early frontend pages assumed a top-level array, so generated content silently rendered as empty. Normalization logic was added at the display layer once identified.

Repository

<https://github.com/Kirneill/StudyBros>

6 Testing, Verification & Evaluation (SO2, SO6)

Verification is layered to match the architecture: backend logic is covered by an automated pytest suite, API contracts are checked against the actual frontend types (not just backend assumptions), and the web app was independently evaluated for accessibility, performance, and UX correctness through structured audits. Every backend change was gated on three checks before being considered done: `pytest -q`, `ruff check .`, and `mypy study_guide api mcp_server`; frontend changes were gated on `npm run lint` and `npm run build`.

File	Tests	Covers
<code>Test_api.py</code>	40	FastAPI routes documents, generate, study, gamification, upload, export
<code>Test_mcp_server.py</code>	40	MCP tools, resources, and prompts
<code>Test_gamification.py</code>	22	Phase tracking, achievements, streaks, consistency
<code>test_scheduler.py</code>	22	FSRS v5 scheduling logic
<code>test_export.py</code>	16	JSON, Anki CSV, Markdown export
<code>Test_generation.py</code>	16	Structured-output generation schemas and prompts

test_database.py	14	CRUD operations across all models
test_learning_models.py	11	CardReview, StudySession, UserProgress, Achievement models
test_extraction.py	11	PPTX, PDF, TXT/MD extraction
test_integration.py	10	End-to-end ingest → generate → export flow
test_chunking.py	7	Paragraph/heading-based and fixed-size chunking
Total	211	

Beyond unit tests, the team ran the API against a temporary SQLite database with a real uvicorn process and made live HTTP requests to confirm the contract worked end-to-end, not just in mocks — covering health checks, document listing, study set listing, due-card retrieval, review submission, scheduling, and all three gamification endpoints. Upload reliability was verified live as well, including successful uploads of real zyBooks PDFs at 16.8 MB and 21.9 MB through the frontend proxy path, and a confirmed diagnostic (rather than generic) error on a malformed PDF.

One specific verification practice worth calling out: the team treated frontend/backend type mismatches as a testable contract, not just a manual check. When the /gamification/consistency endpoint's shape drifted from what the frontend expected, the fix included adding regression tests asserting the exact response keys for /gamification/phase, /gamification/strengths-weaknesses, and /gamification/consistency — so a future drift would fail a test instead of crashing a page.

Two structured audits were run against the frontend, evaluating accessibility (WCAG 2.1 AA), performance, resilience, design consistency, and UX flow:

- An initial audit across 20 pages and 20 components identified 42 findings (8 critical, 16 high, 18 low) — including an inaccessible comparison table, a client-rendered layout forcing unnecessary re-renders on every navigation, and hardcoded fake accuracy stats on the session-completion screen.
- A deeper follow-up audit, scoped against the frontend-ui-engineering skill and a full WCAG 2.1 AA checklist across 42 files, found 110 findings (21 critical, 52 high, 32 medium, 5 low). It surfaced systemic issues rather than one-off bugs — API errors silently discarded on every page, no aria-live regions on dynamic content (card reveals, quiz feedback, upload progress), no prefers-reduced-motion handling, color used as the sole indicator on several charts, and undersized touch targets on mobile.

Status: both audits are complete and documented; **remediation is not yet complete** — it's tracked as Phase B of the remaining roadmap, blocked behind the landing page overhaul (Phase A). This is an honest gap, not a hidden one: the audits demonstrate the team evaluated the implementation critically, even though not every finding was fixed by the documentation deadline.

Requirement	Status
FR 1 (Ingestion), FR 2 (Chunking)	Met: carried from Phase 1, covered by test_extraction.py, test_chunking.py, live upload tests
FR 3 (Generation, multi-provider)	Met: test_generation.py, live provider-switching verified
FR 4 (FSRS/Mastery), FR 5 (Progress/Gamification)	Met: test_scheduler.py, test_gamification.py, test_learning_models.py
FR 6 (Export)	Met: test_export.py

FR 7 (Interface: API/Web/MCP/CLI)	Met functionally: test_api.py, test_mcp_server.py; not fully polished per §6.5 audits
NFR 2 (Reliability: diagnosable errors)	Met: verified live with malformed PDF returning stage-specific 422
NFR 4 (Cost control)	Met: enforced pre-call, unit tested via generation tests
NFR 7 (Security: key handling)	Partially met: server-side .env keys are not persisted client-side as designed, but the audit found personal API keys are stored in browser localStorage in plaintext rather than sessionStorage or not at all, which is a real gap against the intended design
NFR 8/9 (Maintainability)	Met: lint/typecheck enforced clean, 211-test suite maintained throughout

7 Software Engineering Evidence (SO6)

This chapter is specific to Computer Science. Student Outcome 6 for your program is: “Apply computer science theory and software development fundamentals to produce computing-based solutions.”. Each section below maps to one or more of the KPR evidence questions in Appendix A — fill in every section, and where the evidence already lives in Chapters 2–6, summarize briefly and point to it.

7.1 Software Development Life Cycle

The project followed an incremental, phase-gated development model rather than a single big-bang build. Each phase was scoped narrowly through the implementation of Scrums and Backlogs, had explicit entry criteria (what had to be true before it could start), and produced a testable artifact before the next phase began, planned through the sprint planning meetings.

Phase	Scope
0	Fix critical issues from the pre-build code review, add FSRS + gamification data models and tests
1	MCP server — tools, resources, prompts
2	LLM-facing documentation (llms.txt) and skills
3	FastAPI backend + gamification endpoints
4	Next.js frontend + gamification UI, deployed to Vercel
5	Interactive study UI, session-completion screen, strengths/weaknesses reporting
6	Polish + production deployment

7.2 Project Specification

Before extending the Phase 1 CLI into a multi-surface platform, the team ran a formal code review of the existing codebase to determine what was safe to build on top of. The verdict was **"Approve with suggestions"** — roughly 1,900 lines of backend code and 1,200 lines of tests, judged as a well-structured 4-layer architecture, but with issues that had to be fixed first:

- **Critical:** no .gitignore existed, meaning a committed .env would leak an API key directly into version control
- **Critical:** a binary SQLite database file (study_guide.db) was tracked in git, risking merge conflicts and stale local state being shared across the team
- **Major:** session management leaks in the CLI — every command opened a database session and closed it manually, so an exception between open and close would leak the connection; fixed by introducing a `get_session_ctx()` context manager used everywhere afterward

This review is the direct ancestor of Phase 0 in the plan above — every phase-0 task traces back to a finding here, which is itself evidence that design decisions were grounded in analysis rather than assumption.

7.3 Feasibility Study and Project Plan

Feature decisions were backed by written research rather than made ad hoc. Two research documents in particular shaped scope:

- A **learning-science review** cataloguing 15 evidence-based study methods (spaced repetition, active recall, interleaving, the generation effect, the "85% rule" for difficulty targeting) — this is what FSRS scheduling and Bloom-level mastery tracking (Ch. 3, 3.4) trace back to
- A **gamification-science review** of 12 evidence-based engagement principles (self-determination theory, flow, mastery progression, habit loops, the overjustification effect) — used explicitly to separate real competency tracking from "engagement theater," which is why the completion-screen fake-stats finding in §6.5 was treated as a correctness bug, not a cosmetic one

7.4 System Walkthroughs

Two structured audits (§6.5) function as formal design/code walkthroughs against defined standards (WCAG 2.1 AA, a performance checklist, and a security checklist) rather than informal self-review — each finding is filed with a file/line reference, an impact statement, and a specific fix, and findings were triaged into a dependency-ordered fix plan (plan.md) before any remediation work started, to avoid rework from fixing shared components out of order.

8 Professional, Ethical & Legal Considerations (SO4)

The most concrete ethical/legal gap in this project is the one your own audit found: personal LLM API keys entered by users in the browser are stored in localStorage in plaintext, under a predictable key prefix. This matters beyond a technical bug — an API key is financial-access credential material (it's billed per use to whoever owns it), and localStorage is readable by any script running on the page, meaning any future XSS vulnerability or malicious browser extension could exfiltrate a user's key silently, with no expiration.

- Design intent vs. reality: the intended design (NFR 7, Ch. 3) was that user-supplied keys be scoped to a single request and not persisted. The audit shows the implementation drifted from that intent.
- Recommended remediation, in order of preference: (1) never persist the key client-side at all — send it per-request and hold it only in memory for the session; (2) if persistence across a session is required for UX reasons, use sessionStorage (cleared on tab close) instead of localStorage; (3) at minimum, give users an explicit "clear saved key" control and disclose the storage behavior in the UI, rather than storing it silently.
- This is flagged as a known, documented limitation rather than a hidden one — see Ch. 6, §6.6, and it should be listed as a limitation in the Executive Summary or a "Known Issues" note if your program expects one.

Users upload their own course material — lecture slides, PDFs, and in some cases recorded video/audio of live lectures. Two considerations follow from that:

- Third-party content in uploads: a recorded lecture may contain a professor's voice, likeness, or copyrighted slide content that the student didn't author. The system transcribes and stores that content (via Whisper) without any explicit consent or attribution mechanism from the original speaker. This is a reasonable use case for personal study, but it's worth stating plainly rather than assuming it away — the tool trusts the uploader to have a legitimate right to use the material for personal study purposes; it does not verify that.
- Retention: there is currently no user-facing data deletion/export flow beyond deleting individual documents and study sets from the UI. There is no documented data-retention policy (how long transcripts, generated content, or review history persist, or whether they're deleted if a user stops using the tool).

Generation depends on OpenAI, Anthropic, and/or OpenRouter, and transcription depends on OpenAI Whisper. Each of these is governed by its own terms of service, acceptable-use policy, and data-handling agreement — content sent to these APIs (including a student's uploaded lecture material) is subject to that provider's policies, not the team's. Users bringing their own API key (rather than using a server-configured one) are individually bound by whatever agreement they have with that provider. This should be disclosed to users, not assumed to be understood.

AI-generated flashcards, quiz questions, and practice tests can be wrong — hallucinated facts, mischaracterized relationships between concepts, or misleading distractors in multiple-choice options are all possible failure modes of LLM generation, even with schema-validated structured output. Structured output guarantees the shape of the response is correct; it says nothing about whether the content is correct. This has real academic stakes: a student relying on an incorrect flashcard to study for an exam is a direct harm, not a hypothetical one. The system currently has no fact-verification step, no citation back to the source chunk it was generated from, and no user-facing disclaimer about this limitation — all reasonable things to note as a limitation or future work item.

The 110-finding UI/UX audit (Ch. 6) is directly relevant here, not just as a QA exercise. A study tool that a screen-reader user, keyboard-only user, or motion-sensitive user cannot use is exclusionary by outcome even if that wasn't the intent — critical findings included undersized touch targets, no prefers-reduced-motion support, and color used as the sole indicator on mastery/progress visualizations. Treating this as an ethical consideration (equitable access to an educational tool) rather than only a technical one is the more defensible framing for this section.

The project is released under the MIT License, chosen as appropriate for an academic/open-source capstone deliverable — permissive, allows reuse, and imposes no obligation on downstream users beyond attribution.

Worth a short explicit statement: the tool generates study aids from a student's own uploaded material for personal review — it is not designed or intended to generate graded coursework, take-home assignments, or exam answers for submission. Where that line sits is a judgment call, and it's reasonable for the documentation to state the intended use case explicitly rather than leave it ambiguous.

9 Team & Process (SO5)

StudyBros was built by a four-person team: Kirneill Joseph as team lead, who also owned backend development; Daniel Labrador, who built the frontend; Goncalo Ferreira Nascimento, who handled data handling; and Lenyn Gabriel Murillo Holguin, who focused on optimizations and documentation. The team originally included a fifth member, Jose "Lucas" Casas, who was part of the project during its initial KSpider phase but was no longer part of the team for this build; his prior scope was absorbed by the remaining four.

Communication ran through WhatsApp for day-to-day coordination, and the team held a daily scrum meeting on Zoom at a fixed time, 8–9 PM, every day of the week, without exception. That consistency was a deliberate process choice rather than a formality. A single fixed daily sync gave the team a reliable checkpoint to surface blockers, divide up new work (like the audit-driven fix batches in Chapter 7), and keep backend, frontend, and data-handling work moving in step with each other despite being developed in parallel. The discipline of holding it daily, every day, is itself evidence of process maturity worth calling

out explicitly for SO5; it's easy to schedule a recurring meeting and much harder to actually keep it every single day for a semester.

10 Communication Artifacts (SO3)

<https://capstone.cs.fiu.edu/projects/su26-studybros>

Appendix A — Student Outcomes & CS Evidence Map

Student outcomes assessed in this program:

SO	Outcome (verbatim — ABET CAC criteria, as adopted in the KFSCIS SLO assessment framework)
1	Analyze a complex computing problem and apply principles of computing and other relevant disciplines to identify solutions.
2	Design, implement, and evaluate a computing-based solution to meet a given set of computing requirements in the context of the program's discipline.
3	Communicate effectively in a variety of professional contexts.
4	Recognize professional responsibilities and make informed judgments in computing practice based on legal and ethical principles.
5	Function effectively as a member or leader of a team engaged in activities appropriate to the program's discipline.
6	Apply computer science theory and software development fundamentals to produce computing-based solutions. [Computer Science]

SO6 evidence questions (KPRs) for Computer Science — complete the “where evidenced” column:

KPR	Evidence question (verbatim)	Where evidenced (section §)
1	Students contributed in the design and development of a small-, medium-, or large-scale software system in their project.	§5, §9
2	Students demonstrated understanding of the Software Development Life Cycle in their project.	§7.1
3	Students developed Project Specification in their project.	§7.2
4	Students performed Feasibility Study and/or develop Project Plan in their project.	§7.3
5	Students developed Requirements Documentation in their project.	§3
6	Students developed Design Documentation in their project.	§4
7	Students performed and documented testing and/or evaluation of the implementation in their project.	§6
8	Students performed system walkthroughs in their project.	§7.4

Appendix B — Revision History

Version	Date	Changes
0.1	7/12/26	First draft — problem statement, stakeholders, requirements, and architecture (Ch 2-4), pulling from old KSpider docs

0.2	7/20/26	Added implementation and testing chapters, pulled real test counts and audit findings from the codebase (Ch 5-6)
0.3	7/24/26	Added SDLC evidence, ethical/legal considerations, team & process, and communication artifacts (Ch 7-10)
1.0	7/29/26	Filled in Appendix A/B, finalized poster text, final version submitted to Capstone Virtual Showcase